

PRD V1 Final — Multi-Tenant Feature Management Platform (Catalyst)

Version: 1.1 | Status: Final for Build | Type: College / Portfolio MVP

Template: Atlassian PRD ([source](https://www.atlassian.com/agile/product-management/requirements)) — sections 1-8 follow it 1:1.

> Change from V1.0 (v1.1): Dropped push propagation. No SSE, no polling, no Redis pub/sub, no <5s gate. Flags are fetched when the app loads; changes take effect on the next load/refresh.

Participants	Product Owner/Dev: You; Stakeholders: Faculty, recruiters; Users: Developer, Release Manager
Target release	V2 MVP — 4 weeks from kickoff, localhost + optional live deploy
V2 theme	Deploy once, release gradually: create flag -> target / rollout % per env -> SDK evaluates on load -> change reflected on next refresh -> audit logged

This PDF was exported from v2/PRD.md. The Markdown file remains the source of truth; this PDF is the shareable snapshot for review.

1. Project Specifics

Product: Catalyst — self-hosted, multi-tenant feature flag service with dashboard, evaluation API, and SDKs.

Participants:

- Product Owner / Dev: You
- Stakeholders: Faculty reviewer, recruiters (demo evaluators)
- End users: Developer (integrates SDK), Release Manager (toggles rollouts)

Status: V1 scoped. Ready for implementation.

Target Release: V1 MVP — 4 weeks from kickoff, localhost + optional live deploy.

V1 Theme: Deploy once, release gradually. Full loop: create flag -> target / rollout % per env -> SDK evaluates on load -> change reflected on next refresh -> audit logged.

2. Team Goals and Business Objectives

1	Decouple deploy from release: any flag can be turned OFF instantly without redeploy.
2	Gradual rollout: 0-100% with sticky, deterministic bucketing.
3	Targeted release: attribute rules (userId, email, country, version) with priority order.
4	Env isolation: same flag has independent state in dev / staging / prod.

- | | |
|---|--|
| 5 | Simple fetch-on-load: SDKs get the flag snapshot when the app loads (bootstrap); new values apply on the next load — no real-time fanout. |
| 6 | Portfolio proof: live demo with 2 apps + JS + Python SDK + audit trail. |

Out-of-scope for V1 success: scale, RBAC, SSO, experimentation analytics, real-time propagation.

3. Background and Strategic Fit

Teams coupling deployment to release can't kill a broken feature, can't beta-test, can't do 1% canaries. Result: risky big-bang releases and hotfix deploys.

LaunchDarkly / Unleash / Flagsmith prove the model, but are heavy SaaS. Catalyst is a simplified, self-hostable clone to demonstrate core distributed-systems concepts: evaluation engine, consistent hashing, multi-tenant isolation (org -> project -> env -> flag).

V1 intentionally skips real-time push: flag states are fetched whenever a client loads. Simpler to build, simpler to reason about, and sufficient for gradual-release workflows where the app naturally re-reads state on navigation/refresh.

4. Assumptions & Constraints

Assumptions:

- | | |
|---|--|
| 1 | Boolean flags only in V1 (<code>true/false</code>). String/JSON variants deferred to Phase 2. |
| 2 | Single region, low scale: <10 orgs, <100 flags, <10k eval/min on laptop. |
| 3 | Stack: TypeScript Node.js API + React dashboard + Postgres (truth). Redis optional as a cache layer — no pub/sub. Rationale: one language, easy hosting, minimal moving parts. |
| 4 | Auth V1 simple: email/password + JWT for dashboard, <code>sdk_key</code> (read-only, per project+env) for evaluation. No RBAC — all logged-in members have full access within org. |
| 5 | SDK consumers are server-side / trusted internal web apps. |
| 6 | Offline SDK never crashes app — falls back to cached value -> <code>defaultValue</code> . |
| 7 | Propagation model: SDK fetches the flag snapshot during <code>init</code> (on app load). No polling, no SSE. A toggle takes effect on the next load/refresh. |

Constraints (college): 1 dev, 4 weeks, must run on free tier (Render/Railway + Vercel or all-local via Docker Compose).

5. User Stories (V1 Functional Requirements)

Evaluation order for all stories:

1. Global kill OFF -> return default
2. Rules by priority -> first match wins
3. % rollout via hash
4. default

Response always: { value: bool, reason: "disabled|rule:<id>|rollout|default", version: N }.

US-1 — Org / Project / Env

As a developer, I want orgs with projects and dev/staging/prod envs so tenants are isolated.

- Accept: create org -> project auto-creates 3 envs; flag key unique per project+env; SDK key scoped to project+env; switching env never leaks state.

US-2 — Flag CRUD + Kill Switch

As a release manager, I want to create, toggle, archive new-checkout so I can kill bad releases.

- Accept: fields key `^[a-z0-9-_.]+$`, name, description, defaultValue; toggle per env instant; archived excluded from bootstrap/evaluate; validation error on duplicate key.

US-3 — Percentage Rollout

As a PM, I want a 0-100% slider so I can do 1% -> 50% -> 100%.

- Accept: slider + numeric input; sticky formula `murmur(hash(flagKey + ":" + userId)) % 100 < percentage`; same user same result across 1000 evals; 0% = OFF, 100% = ON for non-targeted users.

US-4 — Targeting Rules

As a PM, I want IF email endsWith @test.com OR country==IN AND version>=2.0 THEN true so I can beta-test.

- Accept: multiple rules per flag/env with drag-reorder priority; conditions equals, in, notIn, endsWith, startsWith, >=, <, exists; AND within rule, OR across rules configurable; first match wins; playground tester shows matched rule.

US-5 — Evaluate + Bootstrap API

As a developer, I want APIs so my app checks flags fast.

- Accept:
- POST /api/v1/evaluate {flagKey, context:{userId,email,country,version}} -> {value, reason}
- GET /api/v1/bootstrap?env=prod (sdk_key header) -> {flags:[{key,value}], version}
- p95: server eval <150ms, SDK local <10ms post-bootstrap; missing context fields -> rule safely skips, falls to rollout/default.
- **No SSE / streaming endpoint in V1.** Flags are pulled on load; 304 (If-None-Match on version) lets a re-load skip the download when nothing changed.

US-6 — SDKs (JS + Python) + Offline Safety

As a developer, I want isEnabled() that fetches flags on load and never crashes offline.

- Accept JS + Python: init({sdkKey, env}) -> bootstrap once -> isEnabled(key, context) evaluated from memory; memory+disk cache so a reload still works offline; fallback to defaultValue on any network

failure. No crash on network loss.

- **No polling, no SSE, no onUpdate** — values refresh on the next `init`/page load.

```
await catalyst.init({ sdkKey });
catalyst.isEnabled('new-checkout', { userId: 'u123' });
```

US-7 — Dashboard + Playground

As a PM, I want to operate without code.

- Accept pages: org switcher, flag list (search + status dot + env summary), flag detail (per-env tabs, kill toggle, % slider, rules CRUD + reorder, playground `userId/email -> result+reason`, history tab), API keys page (copy/rotate). Non-dev can toggle in demo <30s.

US-8 — Audit Log (light)

As a developer, I want `who/when/old->new` so I can debug.

- Accept: append-only `timestamp, actor, flag, env, action, before_json, after_json` on every mutation; read-only timeline in flag detail; no edit/delete API.

V1 Success Metrics: kill reflected in the 2 demo apps on their next load; sticky 100%; distribution +/-2% at 10k users; 100% mutations audited; org->flag setup <2min.

6. User Interaction and Design

Low-fi sufficient for V1 (Excalidraw/Figma link to be attached):

1	Flag list: [●] new-checkout prod 25% staging ON dev ON [toggle]
2	Flag detail: tabs dev staging prod; top Kill [ON/OFF]; Rollout [slider]; Rules [list + Add rule modal: attribute/operator/value/serve]; Playground [inputs -> Output: true via rule#2]; History [timeline]
3	Keys: sdk_key prod: cp_prod_xxx [copy][rotate]

UX rule: every control shows `reason` — user always knows **why** a flag evaluated true.

Non-functional V1: Postgres source of truth; optional Redis as plain read cache (no pub/sub); bcrypt + JWT; `server_key` never in browser; versioned config `version++` per mutation used solely as a bootstrap cache-buster (ETag/304), not for push.

Data model V1: `organizations(id), projects(org_id), environments(project_id, name, version), flags(project_id, key, default), flag_env_state(flag_id, env, enabled, percentage), targetting_rules(flag_id, env, priority, conditions_json, serve), audit_logs, api_keys(project_id, env, type, hash)`

7. Questions (Resolved for V1 + Residual)

Resolved with defaults (reopen if you disagree):

- Q1 Audit: **IN V1 light** (required by brief) — full export/search Phase 2.

- Q2 Types: **boolean-only V1** — string/JSON Phase 2.
- Q3 SDKs: **JS + Python V1** — browser + server proof.
- Q4 Propagation: **fetch-on-load only** — no SSE, no polling, no push fanout. Latency of a change = time until the app next boots/refreshes. Phase 2 optionality: SSE/polling later.
- Q5 Deploy: **Docker Compose local mandatory, live link stretch** (api, web, postgres; Redis optional).
- Q6 Timeline: **Wk1 Auth+CRUD, Wk2 rules+eval+hash tests, Wk3 bootstrap+JS SDK+UI, Wk4 Python+audit+2 demo apps+video.**

Residual risks: hash skew -> murmur + tests cover; key leak -> rotation covers; stale values after a toggle -> next load picks them up (documented, by design).

8. What We're NOT Doing in V1

Real-time propagation (SSE/polling/WebSockets), RBAC/SSO/SAML, approvals, multivariate/experiments & analytics, scheduling/expiry, flag dependencies, mobile/edge SDKs, OpenFeature, Terraform, multi-region HA, SLA/SOC2. All explicitly Phase 2. Any request for these during build is rejected to protect 4-week scope.

Appendix: Build Order

US-1 + US-2 -> US-5 eval skeleton -> US-3 + US-4 engine -> US-8 audit hook -> US-6 SDKs + bootstrap -> US-7 UI polish -> demo apps.